PCOS project

Praveen Sinha

2026-04-12

load and prepare data

```
df <- read.csv("~/Desktop/DS 4420/PCOS_data.csv")
```

Clean column names

```
colnames(df) <- make.names(colnames(df))
```

Create target

```
df$PCOS <- df$PCOS..Y.N.
```

Drop unwanted columns

```
df <- df[, !(names(df) %in% c(
  "Sl..No",
  "Patient.File.No.",
  "PCOS..Y.N.",
  "X"
))]
```

Convert all columns to numeric

```
df[] <- lapply(df, function(col) {
  if (is.factor(col) || is.character(col)) {
    as.numeric(as.factor(col))
  } else {
    col
  }
})
```

Remove rows with NA

```
df <- na.omit(df)
```

Split features and target

```
X <- as.matrix(df[, !(names(df) %in% c("PCOS"))])
y <- df$PCOS
```

Add intercept

```
X <- cbind(1, X)
```

```
str(X)
```

```
##  num [1:539, 1:42] 1 1 1 1 1 1 1 1 1 1 ...
##  - attr(*, "dimnames")=List of 2
##   ..$ : chr [1:539] "1" "2" "3" "4" ...
##   ..$ : chr [1:42] "" "Age..yrs." "Weight..Kg." "Height.Cm." ...
```

Scale features

```
X[, -1] <- scale(X[, -1])

## defining the metropolis-hastings and helper functions
sigmoid <- function(z) {
  1 / (1 + exp(-z))
}

log_likelihood <- function(w, X, y) {
  p <- sigmoid(X %*% w)
  sum(y * log(p + 1e-9) + (1 - y) * log(1 - p + 1e-9))
}

log_prior <- function(w) {
  sum(dnorm(w, mean = 0, sd = 1, log = TRUE))
}

log_posterior <- function(w, X, y) {
  log_likelihood(w, X, y) + log_prior(w)
}

metropolis_hastings <- function(X, y, n_iter = 5000, step_size = 0.1) {

  n_features <- ncol(X)
  w_current <- rep(0, n_features)

  samples <- matrix(0, nrow = n_iter, ncol = n_features)

  accept_count <- 0

  for (i in 1:n_iter) {

    w_proposal <- w_current + rnorm(n_features, 0, step_size)

    log_post_current <- log_posterior(w_current, X, y)
    log_post_proposal <- log_posterior(w_proposal, X, y)

    log_alpha <- log_post_proposal - log_post_current

    if (log(runif(1)) < log_alpha) {
      w_current <- w_proposal
      accept_count <- accept_count + 1
    }

    samples[i, ] <- w_current
  }

  cat("Acceptance Rate:", accept_count / n_iter, "\n")

  return(samples)
}
```

Sample

```
samples <- metropolis_hastings(X, y, n_iter = 5000, step_size = 0.05)
```

```
## Acceptance Rate: 0.3876
```

```
burn_in <- 1000
posterior <- samples[(burn_in+1):nrow(samples), ]

# smaller step size
samples2 <- metropolis_hastings(X, y, n_iter = 5000, step_size = 0.03)
```

```
## Acceptance Rate: 0.614
```

```
# larger step size
samples3 <- metropolis_hastings(X, y, n_iter = 5000, step_size = 0.07)
```

```
## Acceptance Rate: 0.2518
```

We evaluated multiple step sizes to assess the sampler performance. As we expected, smaller step sizes resulted in higher acceptance rates (≈0.59), indicating overly conservative proposals, while larger step sizes reduced acceptance (≈0.25), reflecting more aggressive exploration. We selected a step size of 0.05, yielding an acceptance rate of approximately 0.41, which provided a strong balance between exploration and stability. The consistency of posterior estimates across step sizes further supports the robustness of the model.

stable results

```
colMeans(samples[(1000+1):5000, ])
```

```
##  [1] -1.829390733 -0.291338474  0.003663577 -0.087034943  0.115990863
##  [6] -0.007785279  0.772304511 -0.156485209  0.115161415  0.799359394
## [11] -0.127744588 -0.357823522 -0.035514876 -0.275974563 -0.134968875
## [16]  0.080409358 -0.769965494  1.387666775 -0.541129863 -0.727086106
## [21]  0.772282926 -0.394497652  0.215798151 -0.460438657 -0.027462175
## [26]  0.735824270 -0.538151718  0.278645854  0.799993725  0.962216670
## [31]  0.723129708  0.025250153  0.597022820  0.287957738  0.390548161
## [36] -0.138973240 -0.043521495  0.927459698  1.996787371  0.171663262
## [41]  0.090281243  0.211410351
```

```
colMeans(samples2[(1000+1):5000, ])
```

```
##  [1] -1.84682776 -0.20030842  0.03785454  0.03648706  0.24031292  0.01763277
##  [7]  0.56068990 -0.08052453  0.05847334  0.71626373 -0.18059385 -0.38809183
## [13] -0.05972307 -0.15171256 -0.17446331  0.06378014 -0.95933194  1.26716626
## [19] -0.64209022  0.37528530 -0.48982905  0.11106665  0.16738360 -0.47115494
## [25]  0.09442641  0.42821017 -0.48639689  0.35677892  0.76100057  1.08299359
## [31]  0.65832212  0.03122668  0.57268523  0.34479141  0.42158612 -0.12833469
## [37] -0.07962345  0.63442899  2.30730074  0.04048755  0.23422265  0.33293099
```

```
colMeans(samples3[(1000+1):5000, ])
```

```
##  [1] -1.888721893 -0.398745978  0.433516267 -0.263114536 -0.378549710
##  [6]  0.005678362  0.806688749 -0.167873700  0.120439248  0.772789331
## [11] -0.159087120 -0.290825483 -0.246520151 -0.300950931 -0.170717025
## [16]  0.144167463 -0.016886448  0.654588708 -0.634774612  0.260607534
## [21] -0.093427317  0.050300156  0.189185516 -0.417236186  0.041060550
## [26]  0.446474137 -0.769465471  0.411258634  0.888778647  0.938478849
## [31]  0.696176730  0.048672850  0.593638282  0.384882852  0.438239746
## [36] -0.237147676 -0.047085674  1.020306194  2.102849133  0.064309209
## [41]  0.135790137  0.231948098
```

Mean estimates

```
coef_mean <- colMeans(posterior)

# 95% credible intervals
lower <- apply(posterior, 2, quantile, 0.025)
upper <- apply(posterior, 2, quantile, 0.975)

results <- data.frame(
  Feature = colnames(X),
  Mean = coef_mean,
  Lower = lower,
  Upper = upper
)

# Fix intercept name
results$Feature[1] <- "Intercept"

head(results)
```

```
##       Feature         Mean      Lower      Upper
## 1   Intercept -1.829390733 -2.2107264 -1.4656270
## 2   Age..yrs. -0.291338474 -0.8079379  0.1000212
## 3 Weight..Kg.  0.003663577 -0.6699394  0.6475900
## 4  Height.Cm. -0.087034943 -0.6608802  0.4274616
## 5         BMI  0.115990863 -0.3282967  0.8558515
## 6 Blood.Group -0.007785279 -0.4013534  0.3912372
```

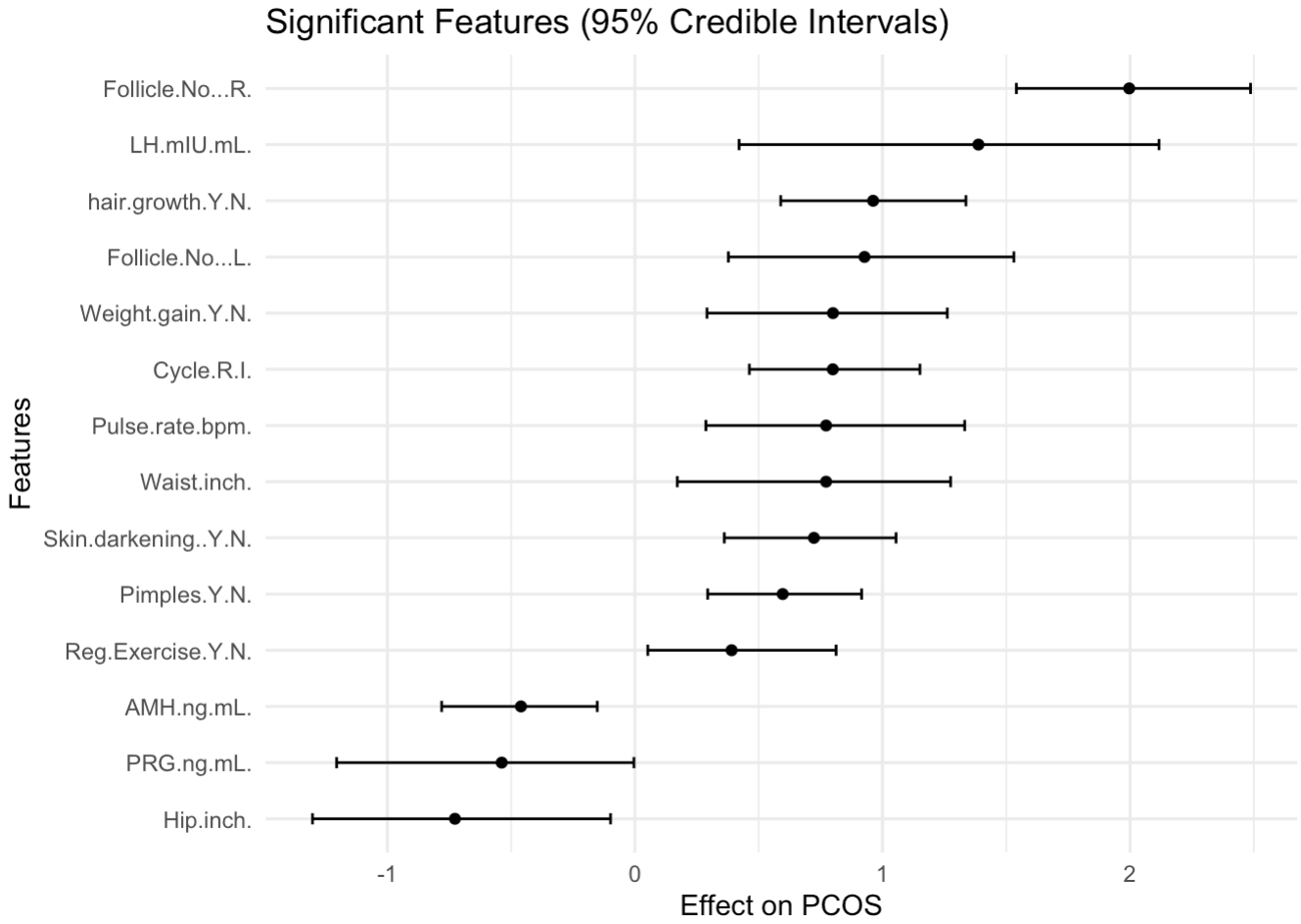
```
important <- results[
  (results$Lower > 0) | (results$Upper < 0),
]

important
```

```
##                 Feature       Mean       Lower        Upper
## 1             Intercept -1.8293907 -2.21072636 -1.465626957
## 7       Pulse.rate.bpm.  0.7723045  0.28662765  1.332198244
## 10           Cycle.R.I.  0.7993594  0.46219519  1.151442895
## 18           LH.mIU.mL.  1.3876668  0.42081069  2.117309002
## 20            Hip.inch. -0.7270861 -1.30243791 -0.098478267
## 21          Waist.inch.  0.7722829  0.17114545  1.275249316
## 24           AMH.ng.mL. -0.4604387 -0.78064527 -0.152181491
## 27           PRG.ng.mL. -0.5381517 -1.20510392 -0.004064406
## 29     Weight.gain.Y.N.  0.7999937  0.29110418  1.261717783
## 30     hair.growth.Y.N.  0.9622167  0.58899363  1.337269174
## 31 Skin.darkening..Y.N.  0.7231297  0.36094701  1.054739313
## 33         Pimples.Y.N.  0.5970228  0.29394553  0.915774077
## 35    Reg.Exercise.Y.N.  0.3905482  0.05208474  0.812768755
## 38     Follicle.No...L.  0.9274597  0.37754825  1.530932152
## 39     Follicle.No...R.  1.9967874  1.54100882  2.487016147
```

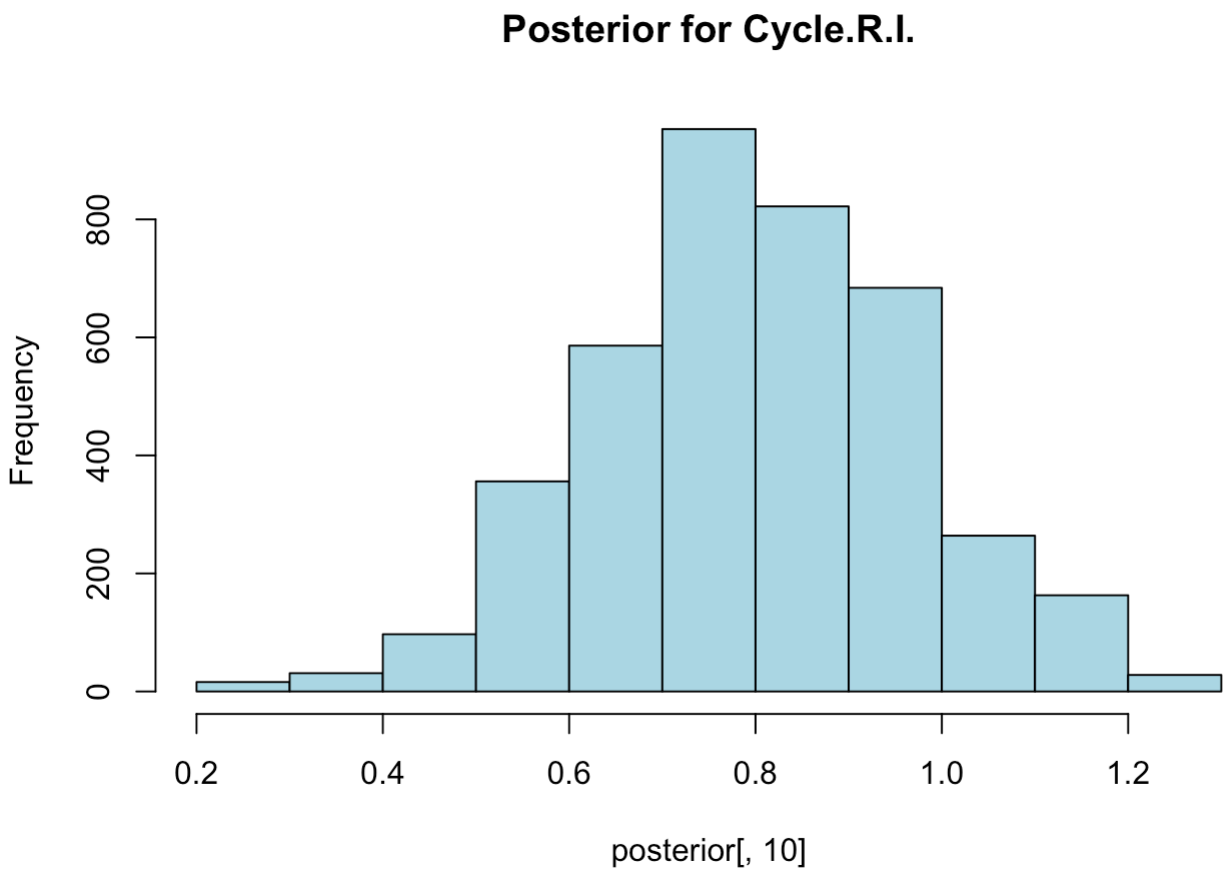
```r
library(ggplot2)

ggplot(important[-1, ], aes(x = Mean, y = reorder(Feature, Mean))) +
  geom_point() +
  geom_errorbar(
    aes(xmin = Lower, xmax = Upper),
    orientation = "y",
    width = 0.2
  ) +
  labs(
    title = "Significant Features (95% Credible Intervals)",
    x = "Effect on PCOS",
    y = "Features"
  ) +
  theme_minimal()
```



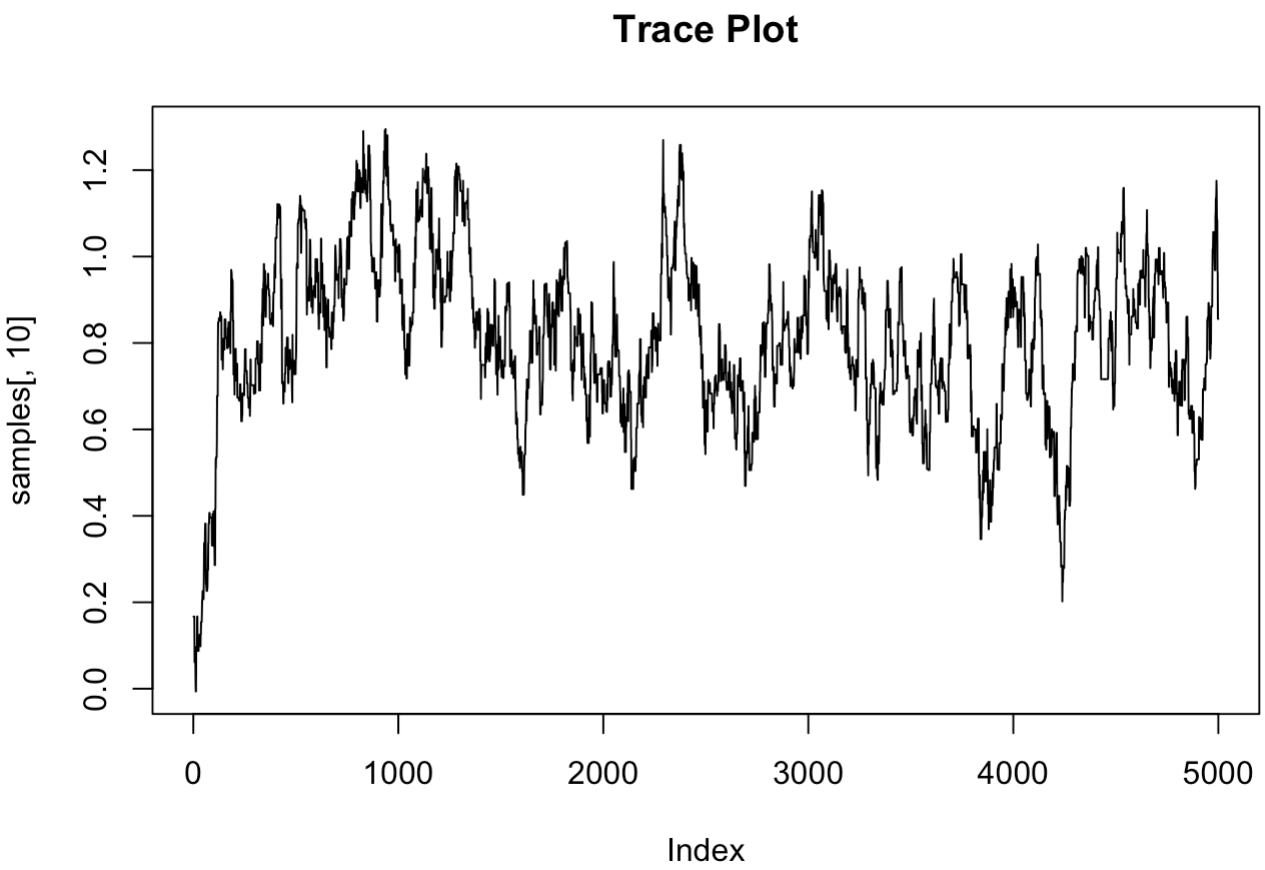
posterior distribution plots

```
hist(posterior[, 10], main="Posterior for Cycle.R.I.", col="lightblue")
```



Trace plot (for convergence)

```
plot(samples[,10], type="l", main="Trace Plot")
```



Coefficients of features

```
for (i in 1:nrow(results)) {
  cat(
    results$Feature[i], ":\n",
    "  Mean =", round(results$Mean[i], 3), "\n",
    "  95% CI = [",
    round(results$Lower[i], 3), ",",
    round(results$Upper[i], 3), "]\n\n"
  )
}
```

```
## Intercept :
##    Mean = -1.829
##    95% CI = [ -2.211 , -1.466 ]
##
## Age..yrs. :
##    Mean = -0.291
##    95% CI = [ -0.808 , 0.1 ]
##
## Weight..Kg. :
##    Mean = 0.004
##    95% CI = [ -0.67 , 0.648 ]
##
## Height.Cm. :
##    Mean = -0.087
##    95% CI = [ -0.661 , 0.427 ]
##
## BMI :
##    Mean = 0.116
##    95% CI = [ -0.328 , 0.856 ]
##
## Blood.Group :
##    Mean = -0.008
##    95% CI = [ -0.401 , 0.391 ]
##
## Pulse.rate.bpm. :
##    Mean = 0.772
##    95% CI = [ 0.287 , 1.332 ]
##
## RR..breaths.min. :
##    Mean = -0.156
##    95% CI = [ -0.521 , 0.161 ]
##
## Hb.g.dl. :
##    Mean = 0.115
##    95% CI = [ -0.266 , 0.591 ]
##
## Cycle.R.I. :
##    Mean = 0.799
##    95% CI = [ 0.462 , 1.151 ]
##
## Cycle.length.days. :
##    Mean = -0.128
##    95% CI = [ -0.447 , 0.173 ]
##
## Marraige.Status..Yrs. :
##    Mean = -0.358
##    95% CI = [ -0.861 , 0.159 ]
##
## Pregnant.Y.N. :
##    Mean = -0.036
##    95% CI = [ -0.464 , 0.397 ]
##
## No..of.abortions :
##    Mean = -0.276
##    95% CI = [ -0.808 , 0.158 ]
```

```
##
## I...beta.HCG.mIU.mL. :
##    Mean = -0.135
##    95% CI = [ -0.375 , 0.16 ]
##
## II....beta.HCG.mIU.mL. :
##    Mean = 0.08
##    95% CI = [ -0.291 , 0.515 ]
##
## FSH.mIU.mL. :
##    Mean = -0.77
##    95% CI = [ -1.835 , 0.082 ]
##
## LH.mIU.mL. :
##    Mean = 1.388
##    95% CI = [ 0.421 , 2.117 ]
##
## FSH.LH :
##    Mean = -0.541
##    95% CI = [ -1.255 , 0.667 ]
##
## Hip.inch. :
##    Mean = -0.727
##    95% CI = [ -1.302 , -0.098 ]
##
## Waist.inch. :
##    Mean = 0.772
##    95% CI = [ 0.171 , 1.275 ]
##
## Waist.Hip.Ratio :
##    Mean = -0.394
##    95% CI = [ -0.814 , 0.025 ]
##
## TSH..mIU.L. :
##    Mean = 0.216
##    95% CI = [ -0.197 , 0.539 ]
##
## AMH.ng.mL. :
##    Mean = -0.46
##    95% CI = [ -0.781 , -0.152 ]
##
## PRL.ng.mL. :
##    Mean = -0.027
##    95% CI = [ -0.484 , 0.3 ]
##
## Vit.D3..ng.mL. :
##    Mean = 0.736
##    95% CI = [ -0.172 , 1.833 ]
##
## PRG.ng.mL. :
##    Mean = -0.538
##    95% CI = [ -1.205 , -0.004 ]
##
## RBS.mg.dl. :
##    Mean = 0.279
##    95% CI = [ -0.164 , 0.699 ]
```

```
##
## Weight.gain.Y.N. :
##    Mean = 0.8
##    95% CI = [ 0.291 , 1.262 ]
##
## hair.growth.Y.N. :
##    Mean = 0.962
##    95% CI = [ 0.589 , 1.337 ]
##
## Skin.darkening..Y.N. :
##    Mean = 0.723
##    95% CI = [ 0.361 , 1.055 ]
##
## Hair.loss.Y.N. :
##    Mean = 0.025
##    95% CI = [ -0.363 , 0.365 ]
##
## Pimples.Y.N. :
##    Mean = 0.597
##    95% CI = [ 0.294 , 0.916 ]
##
## Fast.food..Y.N. :
##    Mean = 0.288
##    95% CI = [ -0.061 , 0.736 ]
##
## Reg.Exercise.Y.N. :
##    Mean = 0.391
##    95% CI = [ 0.052 , 0.813 ]
##
## BP._Systolic..mmHg. :
##    Mean = -0.139
##    95% CI = [ -0.47 , 0.328 ]
##
## BP._Diastolic..mmHg. :
##    Mean = -0.044
##    95% CI = [ -0.469 , 0.379 ]
##
## Follicle.No...L. :
##    Mean = 0.927
##    95% CI = [ 0.378 , 1.531 ]
##
## Follicle.No...R. :
##    Mean = 1.997
##    95% CI = [ 1.541 , 2.487 ]
##
## Avg..F.size..L...mm. :
##    Mean = 0.172
##    95% CI = [ -0.295 , 0.562 ]
##
## Avg..F.size..R...mm. :
##    Mean = 0.09
##    95% CI = [ -0.228 , 0.449 ]
##
## Endometrium..mm. :
```

```
##    Mean = 0.211
##    95% CI = [ -0.111 , 0.655 ]
```

The strongest predictor in our model was follicle count, particularly in the right ovary, which is directly aligned with the Rotterdam criteria for PCOS diagnosis.

all results

```
write.csv(results, "bayesian_manual_coefficients.csv", row.names = FALSE)
```

important

```
write.csv(important, "bayesian_important_features.csv", row.names = FALSE)
write.csv(results, "~/Desktop/DS 4420/bayesian_manual_coefficients.csv", row.names =
FALSE)
```

train/test split

```
set.seed(42)

idx <- sample(1:nrow(X), 0.8 * nrow(X))

X_train <- X[idx, ]
y_train <- y[idx]

X_test <- X[-idx, ]
y_test <- y[-idx]
```

Posterior averaging

```
samples <- metropolis_hastings(X_train, y_train, n_iter = 5000, step_size = 0.05)
```

```
## Acceptance Rate: 0.4558
```

```
burn_in <- 1000
posterior <- samples[(burn_in+1):nrow(samples), ]
```

```
predict_proba <- function(X_new, posterior_samples) {
  probs <- apply(posterior_samples, 1, function(w) {
    1 / (1 + exp(-X_new %*% w))
  })

  rowMeans(probs)
}
```

train accuracy

```
train_probs <- predict_proba(X_train, posterior)
train_preds <- ifelse(train_probs > 0.5, 1, 0)

train_accuracy <- mean(train_preds == y_train)

train_accuracy
```

```
## [1] 0.9327146
```

test accuracy

```
test_probs <- predict_proba(X_test, posterior)
test_preds <- ifelse(test_probs > 0.5, 1, 0)

test_accuracy <- mean(test_preds == y_test)

test_accuracy
```

```
## [1] 0.9074074
```

confusion matrix

```
table(Predicted = test_preds, Actual = y_test)
```

```
##          Actual
## Predicted  0  1
##         0 70  8
##         1  2 28
```

Precision + Recall

```
precision <- sum(test_preds == 1 & y_test == 1) / sum(test_preds == 1)
recall <- sum(test_preds == 1 & y_test == 1) / sum(y_test == 1)

precision
```

```
## [1] 0.9333333
```

```
recall
```

```
## [1] 0.7777778
```

F1 score

```
# PCOS = 1 (positive class)
precision_pos <- sum(test_preds == 1 & y_test == 1) / sum(test_preds == 1)
recall_pos <- sum(test_preds == 1 & y_test == 1) / sum(y_test == 1)

f1_pos <- 2 * (precision_pos * recall_pos) / (precision_pos + recall_pos)

f1_pos
```

```
## [1] 0.8484848
```

```
# PCOS = 0 (negative class)
precision_neg <- sum(test_preds == 0 & y_test == 0) / sum(test_preds == 0)
recall_neg <- sum(test_preds == 0 & y_test == 0) / sum(y_test == 0)

f1_neg <- 2 * (precision_neg * recall_neg) / (precision_neg + recall_neg)

f1_neg
```

```
## [1] 0.9333333
```

log-loss entropy

```
log_loss <- function(y_true, y_prob) {
  eps <- 1e-9
  y_prob <- pmin(pmax(y_prob, eps), 1 - eps)

  -mean(y_true * log(y_prob) + (1 - y_true) * log(1 - y_prob))
}

loss <- log_loss(y_test, test_probs)

loss
```

```
## [1] 0.3178085
```

AUC

```
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.
```

```
##
## Attaching package: 'pROC'
```

```
## The following objects are masked from 'package:stats':
##
##     cov, smooth, var
```

```
roc_obj <- roc(y_test, test_probs)
```

```
## Setting levels: control = 0, case = 1
```

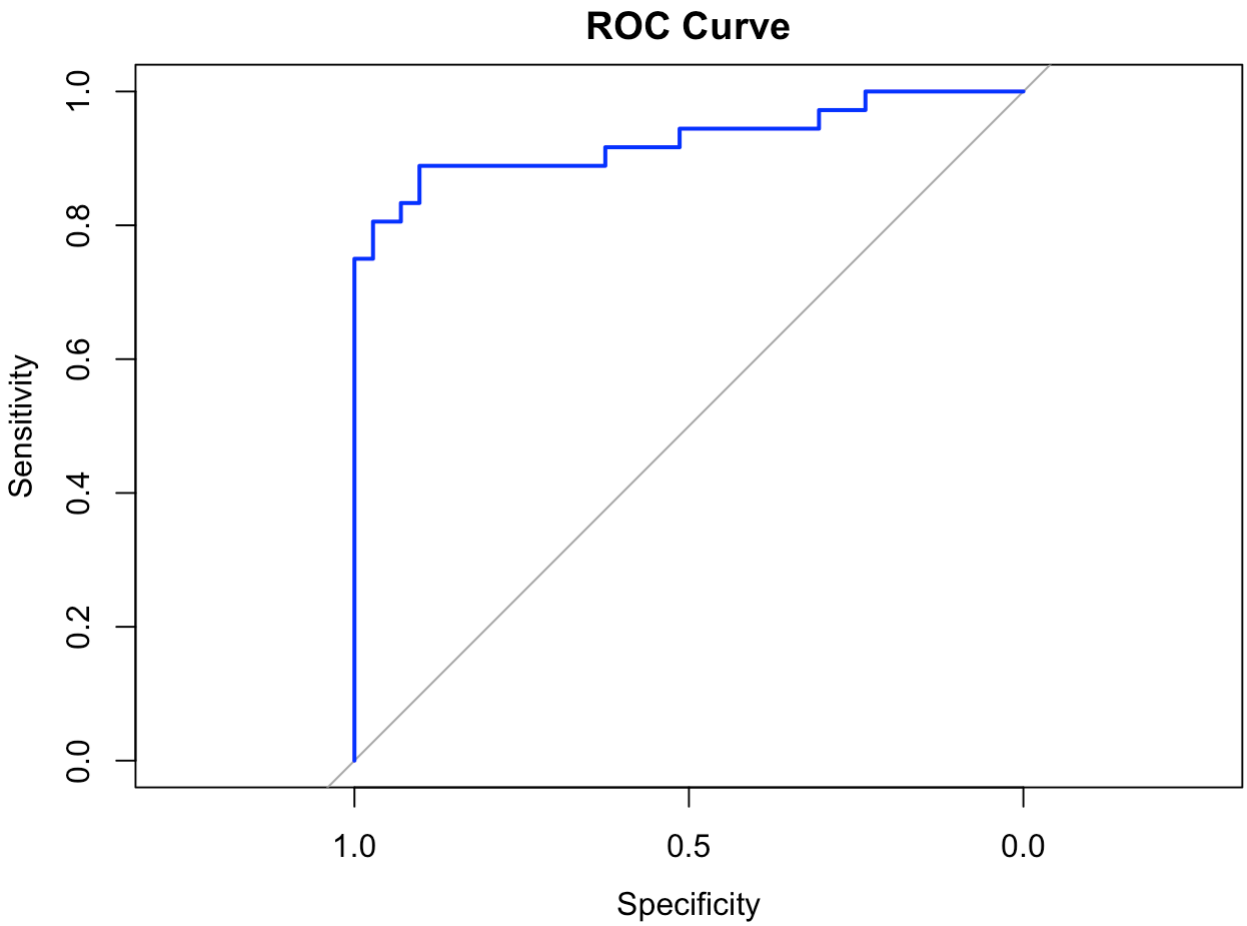
```
## Setting direction: controls < cases
```

```
auc_value <- auc(roc_obj)

auc_value
```

```
## Area under the curve: 0.9267
```

```
plot(roc_obj, col="blue", main="ROC Curve")
```




```
roc(y_test, test_probs)
```

```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
```

```
##
## Call:
## roc.default(response = y_test, predictor = test_probs)
##
## Data: test_probs in 72 controls (y_test 0) < 36 cases (y_test 1).
## Area under the curve: 0.9267
```